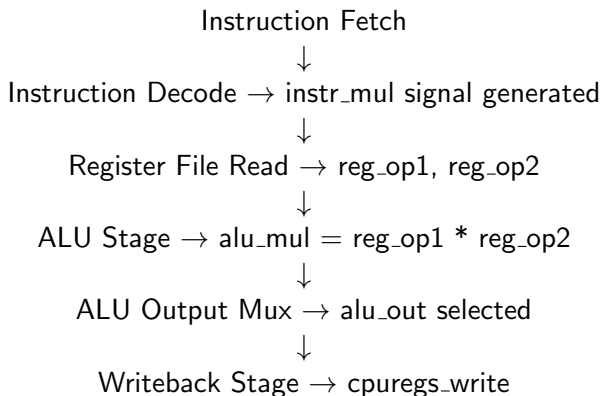


Core-Integrated MUL Implementation in PicoRV32

Datapath of Implementation (Core-Integrated MUL)



Register File Declaration (Line 204)

```
204: reg [31:0] cpuregs [0:regfile_size - 1];
```

This declares the RISC-V register file (x0–x31). Each register is 32 bits wide and stores architectural state.

PC + Operand Registers (Line 177)

```
177: reg [31:0] reg_pc , reg_next_pc ,  
      reg_op1 , reg_op2 , reg_out ;
```

- reg_pc → Program Counter
- reg_op1 → rs1 value
- reg_op2 → rs2 value

MUL Instruction Declaration (Line 654)

```
654: reg instr_mul;
```

Signal that becomes 1 when a MUL instruction is detected.

MUL Decode Logic (Line 1074)

```
1074: instr_mul <= is_alu_reg_reg &&  
      mem_rdata_q[14:12] == 3'b000 &&  
      mem_rdata_q[31:25] == 7'b00000001;
```

- funct3 = 000
- funct7 = 00000001
- opcode = 0110011 (R-type)

RISC-V R-Type Format

funct7	rs2	rs1	funct3	rd	opcode
31-25	24-20	19-15	14-12	11-7	6-0

ADD: funct7 = 0000000

MUL: funct7 = 0000001

Line 887

```
decoded_rd <= mem_rdata_latched[11:7];
```

Line 889

```
decoded_rs2 <= mem_rdata_latched[24:20];
```

Line 932

```
decoded_rs1 <= mem_rdata_latched[19:15];
```

Extracts rd, rs1, rs2 from 32-bit instruction.

Line 1362

```
cpuregs_rs1 = decoded_rs1 ?  
             cpuregs[decoded_rs1] : 0;
```

Line 1363

```
cpuregs_rs2 = decoded_rs2 ?  
             cpuregs[decoded_rs2] : 0;
```

Ensures x0 always returns 0.

Line 1598

```
reg_op1 <= cpuregs_rs1;
```

Line 1605

```
reg_op2 <= cpuregs_rs2;
```

Operands loaded into ALU input registers.

Line 1234

```
reg [31:0] alu_mul;
```

Line 1244

```
alu_mul <= reg_op1 * reg_op2;
```

Performs multiplication.

ALU Output Selection (Line 1289)

```
alu_out = TWO_CYCLE_ALU ?  
    alu_mul :  
    (reg_op1 * reg_op2 );
```

Selects between:

- Registered multiply result
- Direct combinational multiply

Writeback Selection (Line 1331)

```
cpuregs_wrdata =  
latched_stalu ?  
alu_out_q : reg_out;
```

If `latched_stalu = 1` → write ALU result. Otherwise → write other source (e.g., `PC+4`, immediate).

Write Enable (Line 1313)

```
reg cpuregs_write;
```

```
1328: cpuregs_write = 1;
```

```
1332: cpuregs_write = 1;
```

Enables register write for ALU instructions.

Line 1348

```
if (resetsn && cpuregs_write && latched_rd)
```

Line 1354

```
cpuregs[latched_rd] <= cpuregs_wrdata;
```

Writes MUL result into destination register rd.